Reflection and Machine Vision Based Resistor Recognition

Louis Robinson 22007267

Reflection Report

Tutorial 5: *Image Convolution*

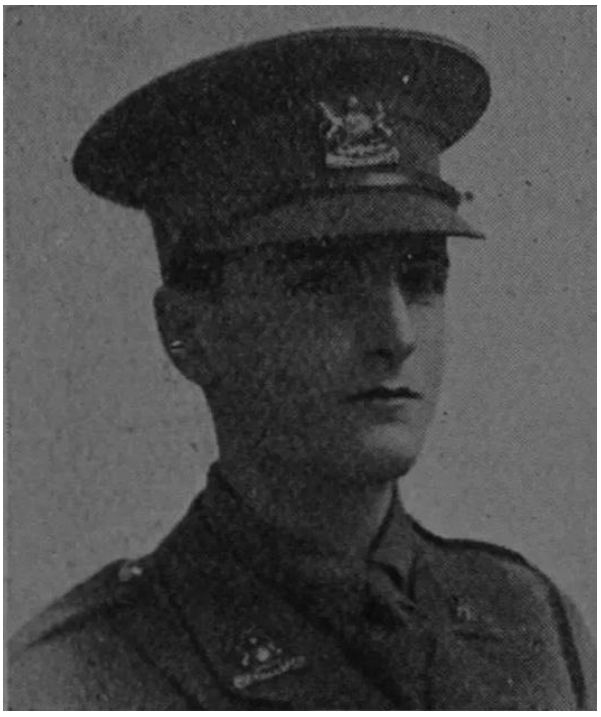


Figure 1: De Ruvigny's Roll Of Honour (UK Photo and Film Archive, 2023)

Types of Noise



Figure 2: Salt and Pepper Noise (S&P)



Figure 3: Gaussian Noise

Salt and Pepper Noise occurs when a pixel's value is lost and replaced with either a 0 (black) pixel, or 255 (white) pixel for an 8-bit image (Zhang K, Crooks E, Orlando A., 2011). This forms speckled distortion of white and black pixels over an image, reducing clarity.

Gaussian noise is random/statistical noise that's intensity follows a gaussian distribution. It is categorised by its mean and variance value, influencing the expected value of the pixel and noise distribution (Iqbal, S *et al.,* 2022)

1. Box Blur (Normalised)



Figure 4: Applying a (3x3) Box Blur to S&P and Gaussian noise

The 3x3 Box Blur kernel takes a 3x3 matrix for each pixel within an image, taking its own and neighbouring pixel colour values. The central pixel's value is then re-calculated through finding the mean value of the matrix. This forms a blur like effect, useful for blending harsh contrast noise in an image like S&P noise, but less so for Gaussian noise.



Figure 5: Applying a (9x9) Box Blur to S&P and Gaussian noise

Expanding the kernel size to a 9x9 matrix, effects become more visible as more pixels dilate the mean colour values accuracy. This creates a more generalised value which leads to higher image distortion (blur).

2. Median Filtering



Figure 6: Applying a (3x3) and (7x7) Median filter to S&P and Gaussian noise

Through finding the median value of the matrix instead the mean, Median Filtering can completely filter out impulse noise. As the median value must equal one of the pixels inside the neighbourhood, it doesn't create new colour values, keeping edges sharper than mean filters.

However, Median filtering is limited to impulse noise only (S&P), as it's only effective if less than half the pixels in a neighbourhood have been compromised.

As we can see, a kernel size of 3x3 perfectly restores our image back to the original whilst 7x7 starts to introduce blur.

3. Sharpening



Figure 7: Applying a (3x3) and (7x7) Median filter to S&P and Gaussian noise

Sharpening emphasises the colour difference between neighbouring pixels, creating better defined edges through contrast (Powell. V, 2017). Through multiplying our 3x3 image matrix by our sharpening kernel, we sum all the values to get our central pixel value. This differs to box blur as we don't divide by the number of pixels.

However, as the kernel increases the contrast, any remaining noise will be amplified. This is demonstrated in Figure 7 when applying the sharpening kernel to gaussian noise.

The resulting images have better defined edges although appear slightly unnatural due to the large changes in contrast.

Tutorial 8: *Training a CNN with CIFAR Images*

During this tutorial the CIFAR Dataset was used to train a CNN. This contains 50,000 training images and 10,000 test images, classifying the images into 10 categories “Airplane, automobile, bird, cat,..ect”.

The CIFAR Dataset is downloaded and images scaled down to 32x32 in order to speed up the CNN execution time.

The convolutional base is then created using convolution and pooling layers to gather trainable parameters before sending it to the dense layer to perform the classification.

Classifying Images Using the CNN:

The model performs well on images that are similar to the CIFAR library that the model was trained on but struggles with conflicting images such as a boat on land (Figure 9).

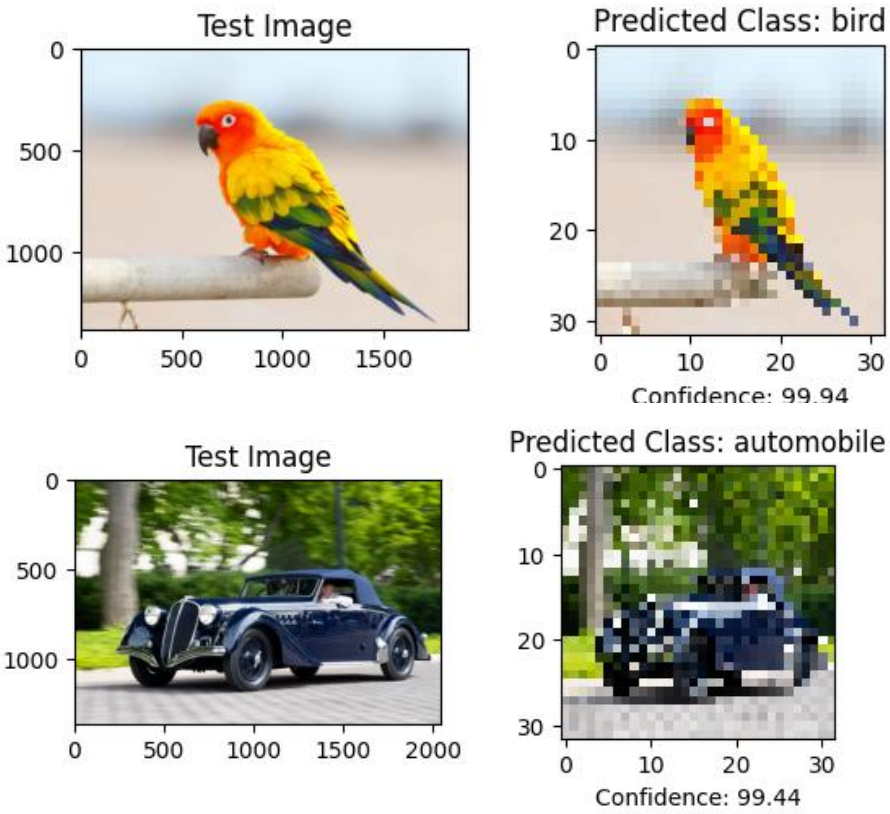


Figure 8: Testing the CNN on easier Images

Furthermore, distortion due to the 32x23 resolution of the image causes finer features to be lost such as deer's antlers, reducing the confidence of the model.

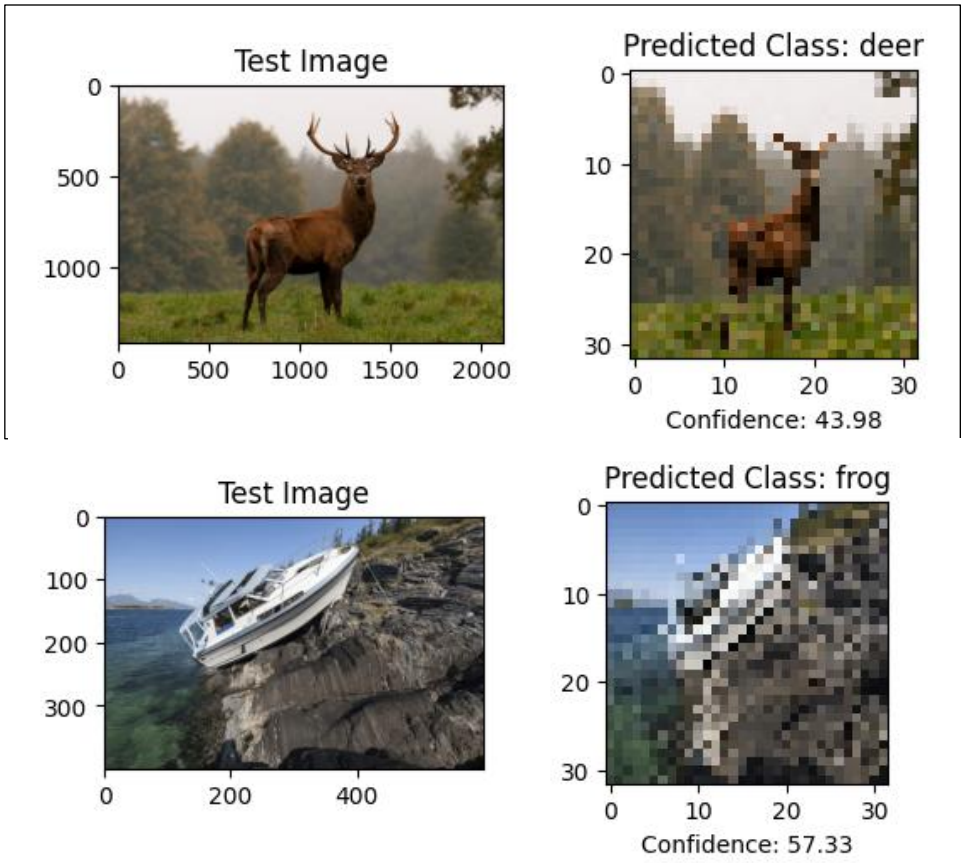


Figure 9: Testing the CNN on harder Images

Improving Performance:

Altering the CNNs structure can allow us to optimise its performance.

Table 1: Alternate Model Structures to test accuracy

Kernels in the first layer	Convolution and pooling layers	Neurons in the hidden dense layer	Number of epoches
8	2	4	5
16	3	32	10
32	4	64	20
64	5	128	30

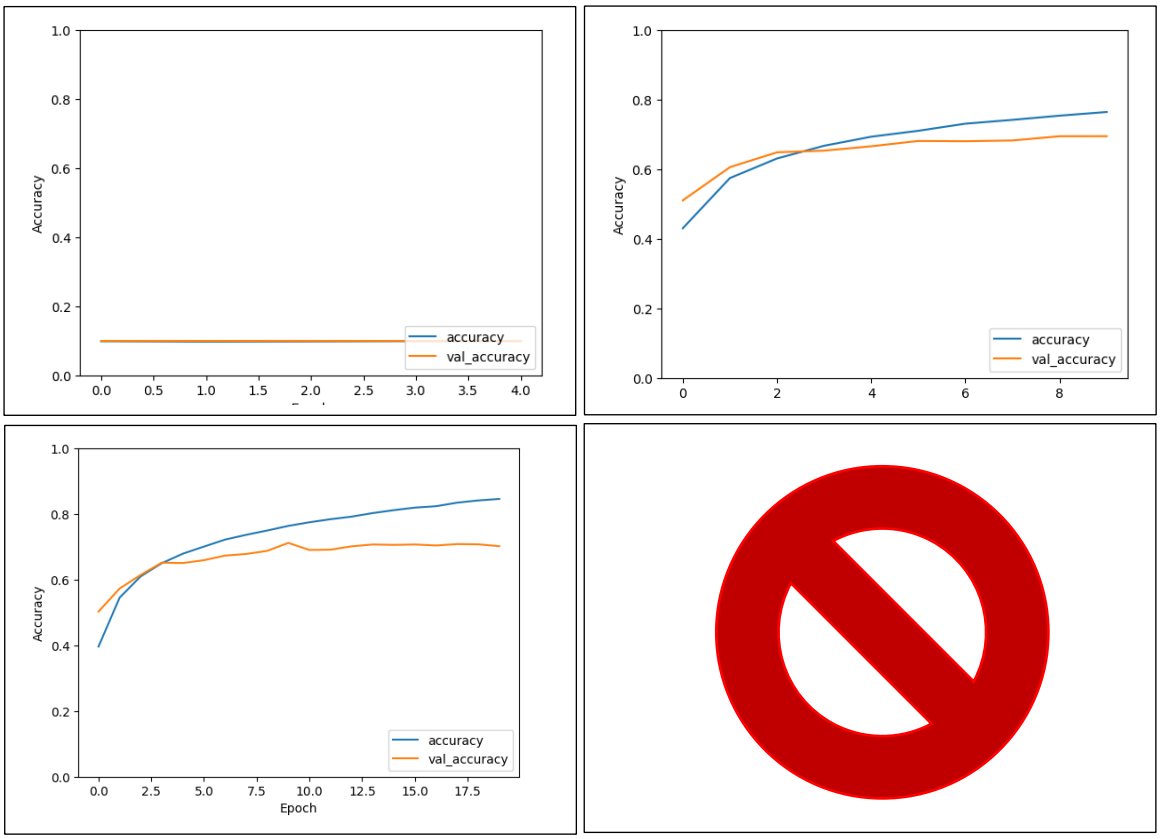


Figure 10: Accuracy results of Model 1 (top left), Model 2 (top right), Model 3 (bottom left), Model 4 (bottom right)

Increasing the depth of the CNN through the number of layers, kernels, neurons and epochs can improve model performance due to an increase of parameters being passed. Figure 10 demonstrates the performance gain between model 1 and 2 going from 10% accuracy to 60%.

However, increasing model depth too far in relation to the input image can cause overfitting and errors to occur. Figure 10 shows that despite an increase in parameters, model 3's val_accuracy does not change. The model is therefore suffering from diminishing returns and overfitting, taking longer to train with no improvement in accuracy.

Model 4 takes this further by erroring due to excessive convolutional and pooling layers, shrinking the feature map past what is possible.

Therefore, out of the four models' model 2 is the most optimised.

Tutorial 9: *Training a Convolutional Neural Network for Image Classification*

Instead of using CIFAR images, a CSV file can be used to read downloaded data sets. Originally a dataset of 40 images was used to train the CNN to differentiate from 0-3, with 9 separate images of each number.

To improve the CNN from 0-3 to 0-9, I used the MINST dataset and created a CSV to store the image labels and categories. The CSV consisted of 120 images, accounting for 10 classes (0-9) with 12 representative images per class.

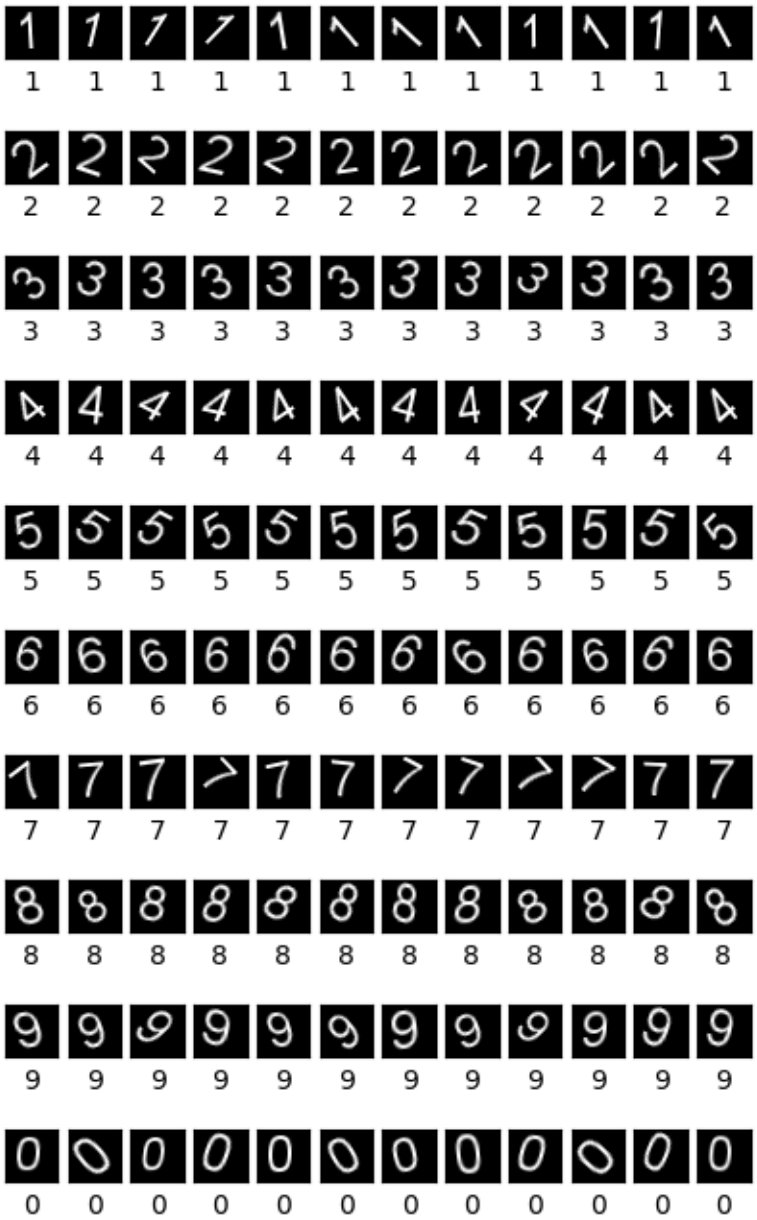


Figure 11: MINST Dataset

To Test the CNN, I randomly selected 12 images from the dataset and labelled them as TEST in the CSV. All other images within the dataset are set to TRAIN which will be used to train the CNN to differentiate between classes.

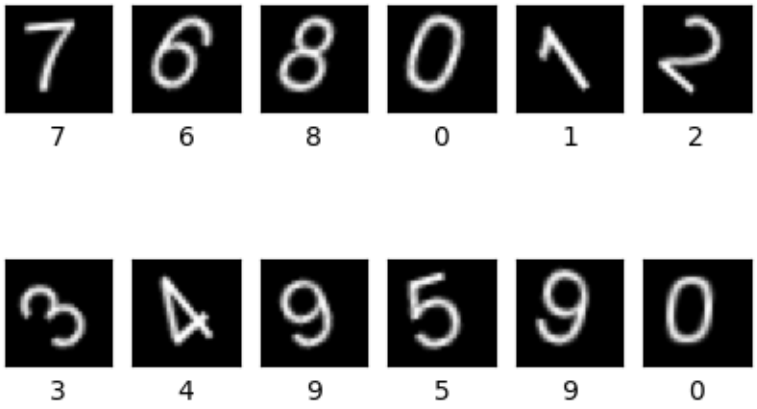


Figure 12: Test images from MINST Dataset

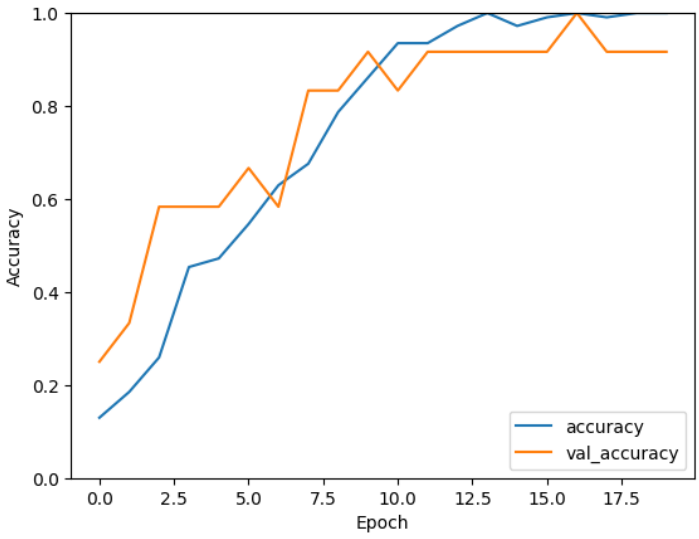


Figure 13: Accuracy of CNN

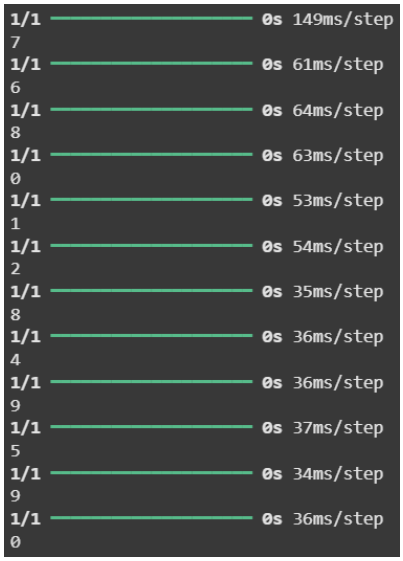


Figure 14: CNN Results on test data

As we can see, the CNN successfully identifies 11/12 of the test images with an accuracy of 91%. The algorithm confuses the 3 to be an 8 most likely due to the image being the most slanted out of its class, varying from its class dataset.

Element B

1.Literature Review

Machine Vision (MV) is a widely researched field exploring object detection and classification within real world settings (Javaid *et al.,* 2022). This report will focus on implementing MV to identify resistor types, simplifying the classification process which is often tedious and prolonged.

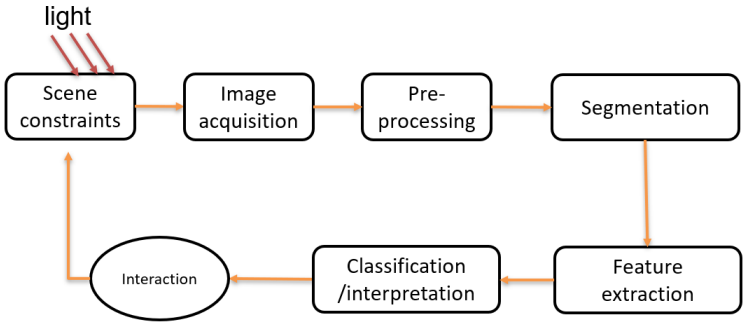


Figure 1: The Traditional Five Stages of Machine Vision (Chen, 2025)

Chen (2025) demonstrates the five traditional stages of MV. Through each of these stages we can evaluate current studies conducted in this field to optimise our resistor recognition algorithm.

The first stage of MV involves Image Acquisition through hardware such as cameras or sensors (Javaid *et al.,* 2022). When acquiring images for datasets, lighting plays an important role in ensuring sufficient detail within an image (Zhou, Zhang and Konz, 2023). With uniform lighting and multiple sources improving performance when compared to singular light sources (Kopparapu, 2006). This helps to maintain the clarity and detail of the resistors which is crucial in identifying their categories as shown in Muminovic and Sokic (2019) through white backgrounds and uniform lighting.

However, uniform backgrounds, lighting and similar resistor types across the dataset such as in Muminovic and Sokic (2019) limit generalisability, impacting the robustness of the algorithm as Deep learning (DL) techniques require adversarial images in training data to maintain accuracy (Trigka and Dritsas, 2025).

Once acquired, images are then pre-processed to enhance and modify the dataset for the specific applications by reducing noise, normalising, scaling, and augmenting. This enables image segmentation to take place, breaking down the image into regions which allow features to be extracted. This can be done through manual methods such as in Muminovic and Sokic (2019) where thresholding and morphological operations were used to segment resistors from the background.

Despite providing a simple method for segmentation, manual methods require low noise and plain bright backgrounds to maintain accuracy and reliability (Muminovic and Sokic, 2019). Often struggling to maintain reliability in diverse datasets due to dynamic resistor size and colour, reducing the accuracy and generalisability of the model.

For this reason, Convolutional Neural Networks (CNN) and DL techniques such as Visual Geometry Group (VGG) are frequently preferred in image analysis (Varna and Abromavičius, 2022). Improving performance in feature extraction when deepening the networks' structure and tailoring the convolutional and dense layers (Liu *et al.,* 2021).

Once the features have been extracted, classification algorithms can be run to interpret and label the object within the image. CNN's compact image features into low-resolution feature maps, learnt by its encoder to classify each pixel within the image (Liu *et al.,* 2021). Loss functions are then used to calculate the error between the models' outputs and the true values, allowing the model to adjust weights and improve its accuracy (Varna and Abromavičius, 2022).

When using DL approaches its important to select appropriate models as they vary in architecture, affecting computational complexity, time, accuracy and generalisability (Trigka and Dritsas, 2025). Varna and Abromavičius (2022) found the highest model accuracy of surface-mounted devices was achieved using YOLOv4 with 97.7% and 98.6% accuracy for resistors. However, this came at the cost of an Interface Time of 1000ms which was joint last with YOLOv3-Keras. This highlights the importance of model selection within application, ensuring selected models can fulfil project requirements such as speed and accuracy.

Other forms of classification are available such as point clouds and support vector machines which Muminovic and Sokic (2019) used to categorise the colour of resistor bands. However, this requires a more statistical interpretation of the resulting data which introduces generalisability and robustness concerns when compared to CNNs.

2. Image Acquisition and Datasets

Image Acquisition is the first step in MV and arguably the most important. If the quality of the attained dataset is poor due to lighting, resolution or generalisability it can negatively affect the results of the algorithm (Zhou, Zhang and Konz, 2023).

For this reason, I decided to use a pre-made Resistor dataset licensed under MIT containing 1270 images of resistors taken via USB microscope. This includes 67 values of 4 band resistors and 60 values of 5 band resistors, with 10 images taken per value. Out of these 10 images, 5 backgrounds are used (wood, white paper, yellow notepad, book cover, book page) with 2 leg orientations (bent and straight) (MIT, 2024).



Figure 2: 4 band 22k resistor with a book page background (MIT, 2024)

Alhanoof et al., (2021) found that dataset representation of the original distribution is key to accurate performance. Therefore, due to the distribution of the dataset covering over 127 resistor types with alternating backgrounds and orientations, it should generalise well to most scenarios.

However, lighting within the dataset can be quite inconsistent, leading to loss of colour detail on images such as Figure 3. Furthermore, glare can be seen on the resistors which could further affect accuracy in feature extraction. Despite being a problem, pre-processing can be used to improve contrast and brightness, providing a quick solution to the lighting of the dataset.



Figure 3: 4 band 100k resistor with a book page background (MIT, 2024)

Figure 4 demonstrates a proposed lighting set up, including multiple light sources to improve image quality and reduce glare based on the results of Kopparapu (2006).

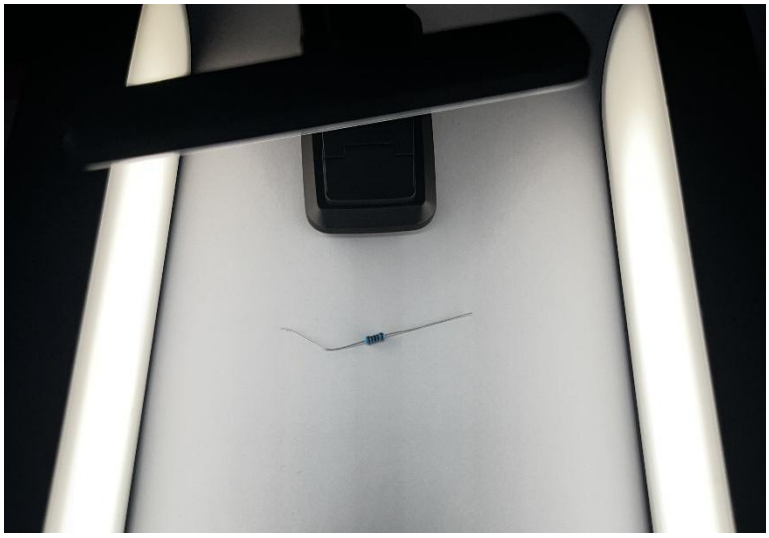


Figure 4: Proposed Light set up to minimise glare and improve lighting

Figure 5 demonstrates the improved image clarity achieved via the lighting set up in Figure 4. Dominant top-down glare has been reduced with the overall brightness of the image improved compared to Figure 3.

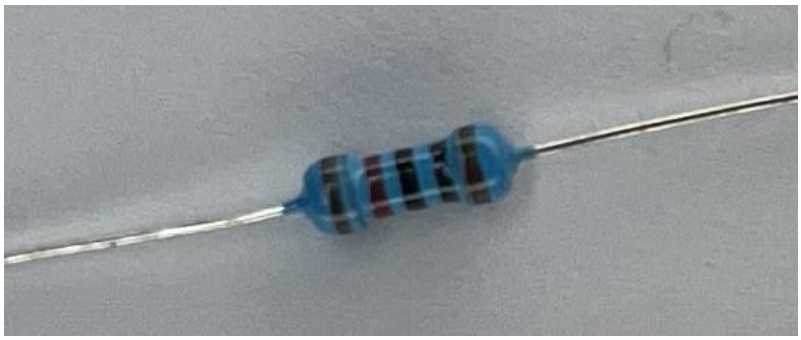


Figure 5: Photo of a 1K resistor using Figure 4 methodology

As I will use two separate CNNs for resistor detection and band classification, different datasets are required to train the models.

For resistor detection 339 images were randomly sampled from MIT (2024), reducing the size of the dataset to preserve execution time. This is then paired with 15 UWE provided images and 65 of my own, including 50 non-resistors and 15 resistors taken via Figure 4 setup, totalling 419 images for the dataset. Out of the 419 images 351 were allocated to train, 58 to testing and 10 to unseen.

For band classification and resistance calculation I again opted to use MIT (2024)'s data set due to its vast resistor coverage and therefore band colour coverage. From the dataset I randomly sampled 112 images and drew on band contours. I then augmented the dataset, randomly sampling to adjust the saturation +-32% and exposure +-10% per image, generating an additional 158 images. Augmentation was done in an attempt to fix the poor lighting of the dataset and improve colour representation in different lighting. The final dataset consists of 270 images divided into 237 training, 22 Validating and 11 Testing.

3. Algorithm design

The four aims of the algorithm are to (1) Identify if a resistor is present within the image, (2) Count the number of bands on the resistor, (3) Recognise the colour of each band, and (4) Calculate the resistance value.

3.1 *Resistor detection*

To identify a resistor within an image, I opted to use a VGG based CNN due to their simple and deep architecture.

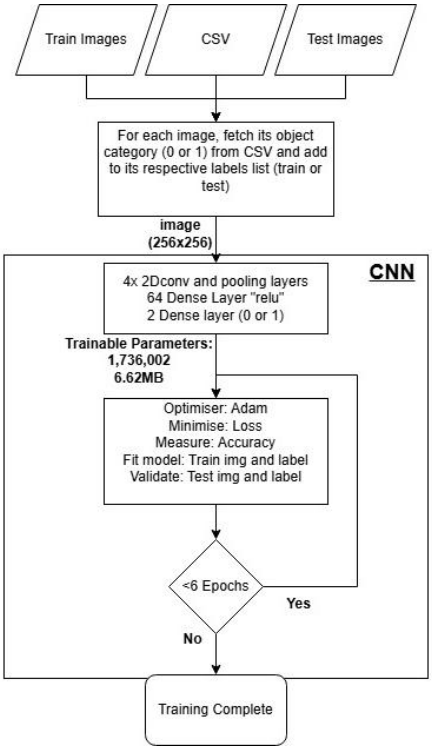


Figure 6: Proposed architecture and training of the resistor detection CNN

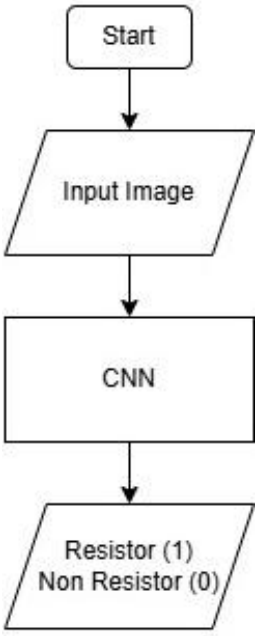


Figure 7: Proposed testing of Unseen images of the resistor detection CNN

As seen in Figure 6, the CNN takes the images within the dataset and stores them based on their category "Test or Train" accessed via the CSV. Once sorted, the algorithm saves the category of the image into 1 (resistor) or 0 (non-resistor) which will be used later for training.

The images are then passed through four 2D convolution and pooling layers, creating 8 layers in total. The dense layers kernel size is then set to 64 and used to categorise the images into two classes 0 or 1. This passes 1,736,002 trainable parameters, enough to reliably distinguish resistors within an image whilst keeping execution time low.

The model implements "adam" as its optimiser with the goal of minimising loss between the training images and the test images.

Due to an input image size of 256x256 to maintain clarity and required detail, a shallower CNN and fewer epochs were opted for to keep the overall training time low and reduce the possibility of overfitting the model from excessive epochs.

Once the model has been trained, Figure 7 demonstrates the procedure to test the model on unseen images to determine if it contains a resistor.

3.2 *Resistor recognition and resistor value calculation*

To classify the number of bands and their colours I decided to use YOLO V11s due to its speed, accuracy on smaller datasets, and overfitting prevention. YOLO has been used in multiple studies such as Varna and Abromavičius (2022) where their YOLOv4 model offered 97.7% and 98.6% accuracy in identifying surface-mounted resistors.

Using websites such as Roboflow, contours can be drawn and labelled on the dataset to provide input training data for the YOLO model, allowing it to classify and segment resistor bands from an image.

The images are then sent to YOLO V11s via the ultralytics package and runs on a Tesla T4 GPU to optimise training times. 80 Epochs were selected with an image size of 640x640 required by YOLO V11s, with the aim of increasing accuracy through maintaining image quality and medium-high epochs.

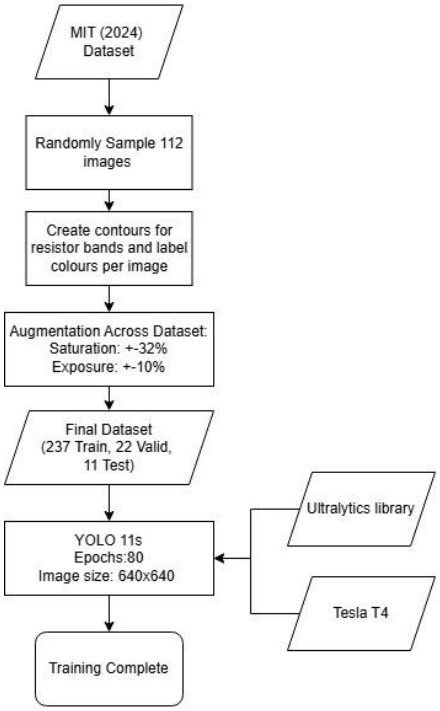


Figure 8: Proposed algorithm and training of the resistor bands classifier via YOLO v11s

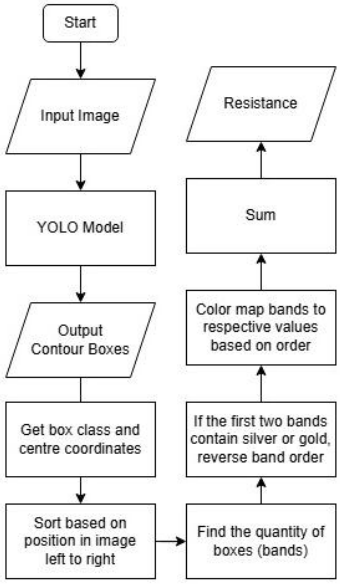


Figure 9: Proposed testing of Unseen images of the YOLO model for resistor band classification

Once bands have been located, functions are used to find the class and centre of each band, sorting them left to right on the resistor. If the first two bands contain silver or gold the image is reversed as the orientation of the resistor is incorrect based on Figure 10.

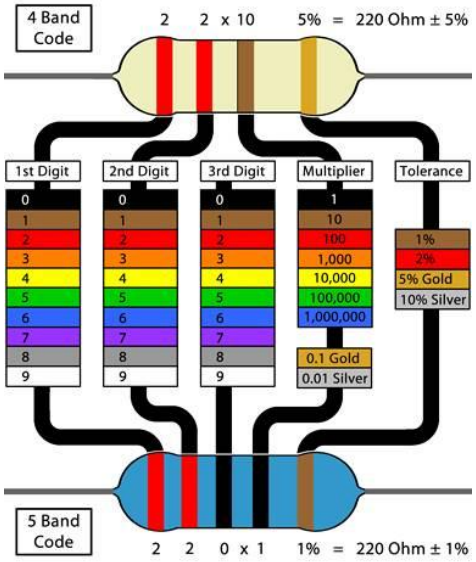


Figure 10: Calculating Resistor Value (42 Electronics, 2025)

Band classes are then colour mapped to their respective values based on colour and position of the band, allowing them to be added or multiplied to find the overall resistance and tolerance.

3.3 *Overall Algorithm*

Therefore, the overall proposed algorithm is shown in Figure 11.

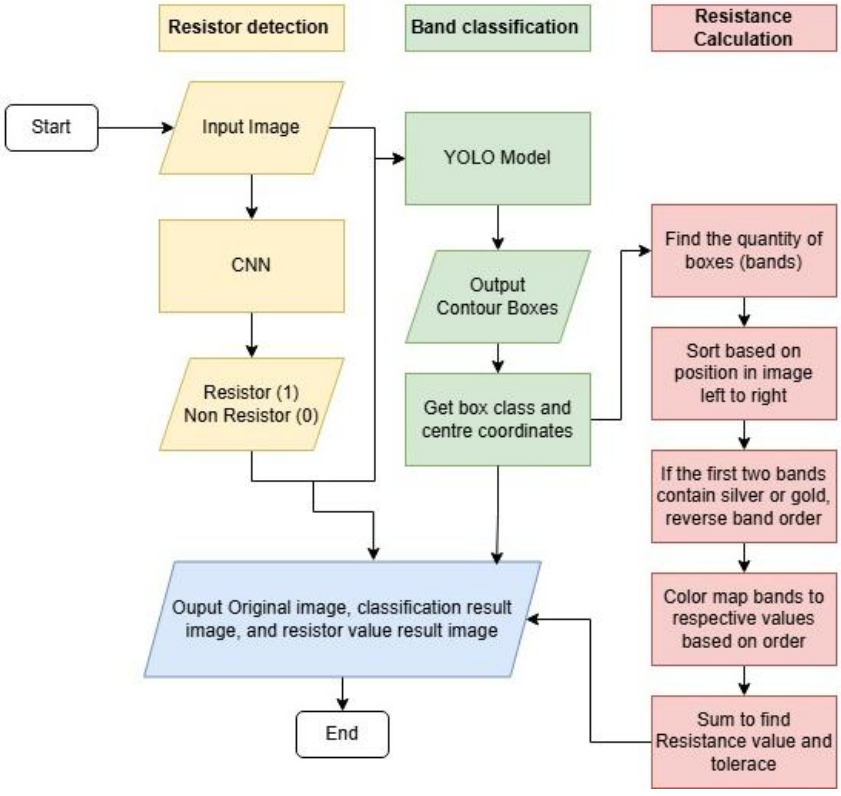


Figure 11: Overall Proposed Algorithm Layout

4. *Implementation details*

In order to implement the CNN and YOLO V11s model into python, several libraries and packages are needed.

For faster general operations such as handling arrays, math and data I decided to implement Numpy. Numpy allows for vectorisation to be possible rather than iterative paradigms. This enables operations to take place on whole arrays or tensors at once rather than individual elements, improving execution time. This is useful in ML when managing big datasets where arrays can contain hundreds of elements.

To read from the CSV's containing all the data on our dataset, the Pandas library is essential for loading this data. Pandas can access the CSV to understand which image belongs to which class. This allows the dataset to be split into training and testing as well as its class (resistor or no resistor) which is required for the training and validation of the resistor identification CNN.

Tensor flow (Tf) Keras is then used to operate and train our CNN, importing the dataset, layers and model. Images are imported via Tf keras to be put into correct formats (float32 or uint8) whilst allowing dimensions to be manipulated ensuring the Tensor is compatible for the model. Using Tf we can create our CNN by importing the type of model we want and defining its convolutional, pooling, and dense layers. We can then select the optimiser we want (adam), the quantity of epochs (6), what we want to measure (accuracy), and fit the model with the data.

Next, the Ultralytics package is required to import and run the YOLO V11s model. It enables the model to be loaded, trained, validated, and results gained, allowing us to find the bounding boxes of the bands and their categories.

I chose to train the YOLO model in a separate notebook and import it via google drive to prevent accidental retraining of the model and gaining different characteristics and charts. Both the resistor identifier CNN and YOLO V11s were tuned in accordance with 3.1 and 3.2 in an attempt to maximise accuracy whilst minimising execution time.

5. *Results*

5.1 Resistor recognition

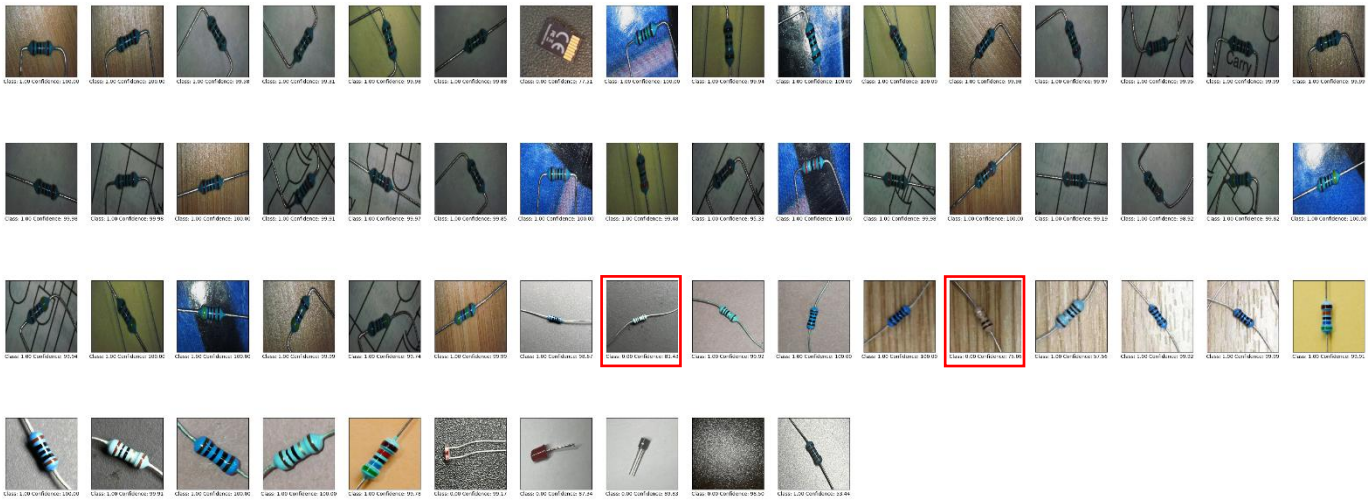


Figure 12: Results testing dataset of the resistor detection CNN

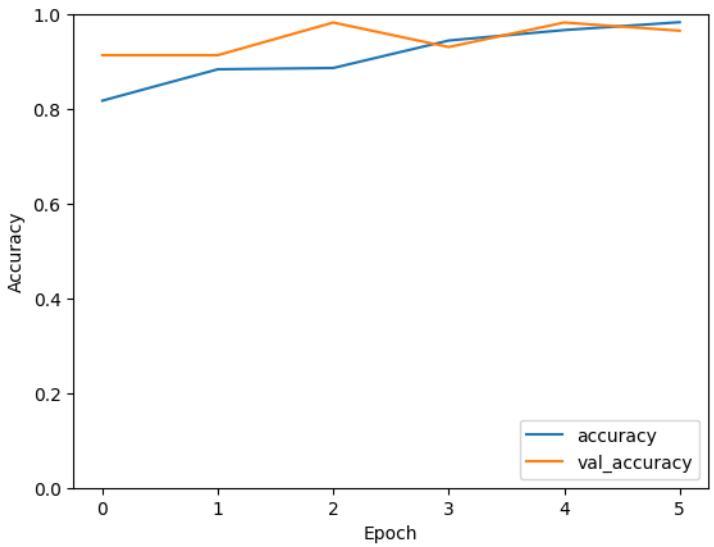


Figure 13: Evaluating models accuracy during training over epochs

The resistor recognition CNN model performs well at recognising resistors within the dataset. Figure 12 shows that out of the 58 test images only two were misinterpreted, providing a test accuracy of 97% (Figure 13).

Errors in classification could be due to the resistor matching the colour of its environment or being too small, creating difficulties in distinguishing in the resistors' features. Ideally this would not be a problem, as provided images in manufacturing or field environments would be of higher quality. Instead, these images are used to test the limits of the model.

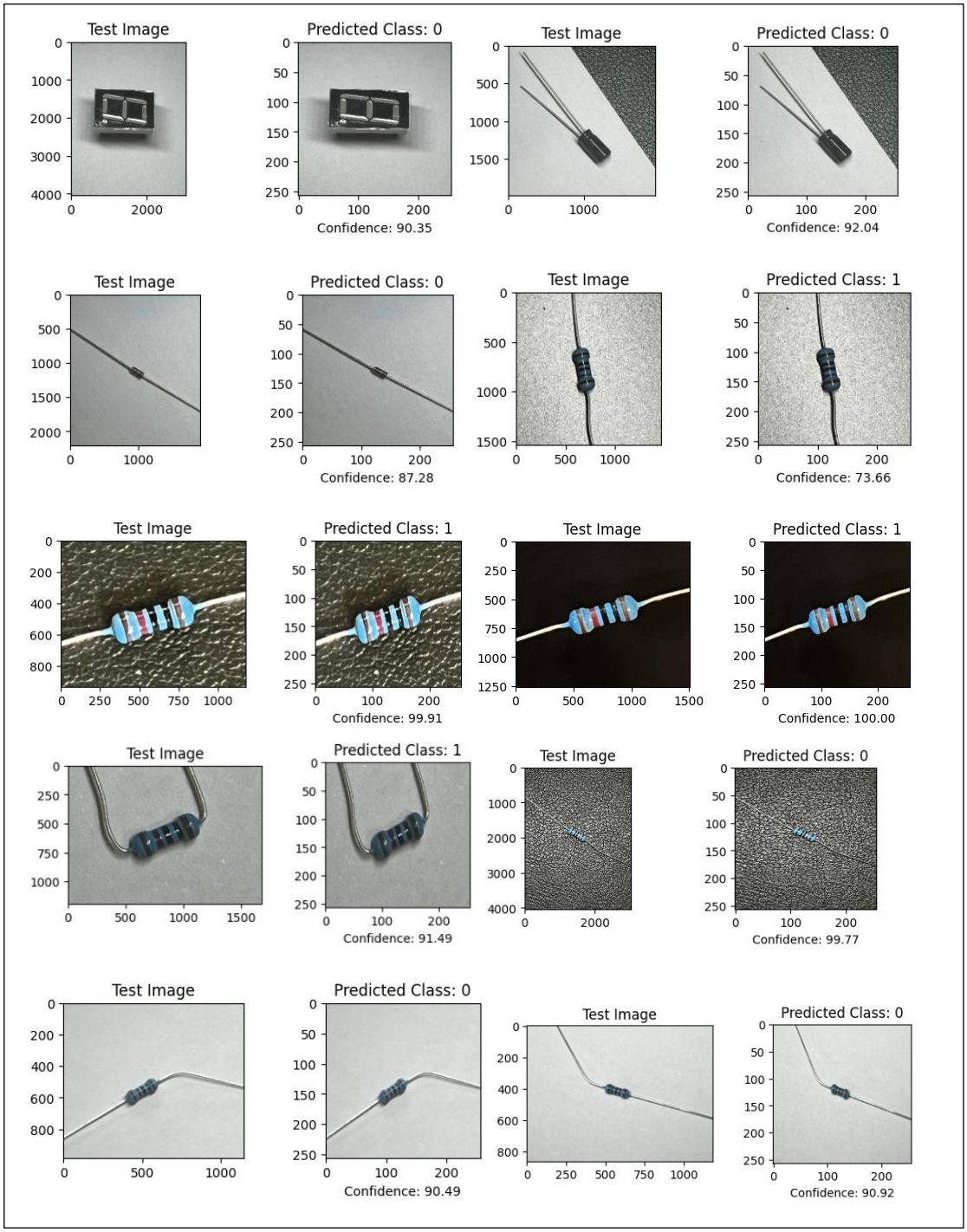


Figure 14: Results testing unseen images with the resistor detection CNN

Testing the model with unseen images in Figure 14 provides us with a different level of accuracy, achieving 70% through successfully classifying 7 out of 10 images. Errors in the prediction follow above mentioned limitations of the model such as size and background.

Overall, the model performs well in identifying non-resistor objects, forming an accuracy of 100% on true negatives but subject to false negatives on background or size variations masking the resistors features.

5.2 Band recognition and resistor value calculation

The YOLO V11s model performed well during the Training and validation stages.



Figure 15: Results testing unseen images with YOLOv11s model after resistor detection CNN

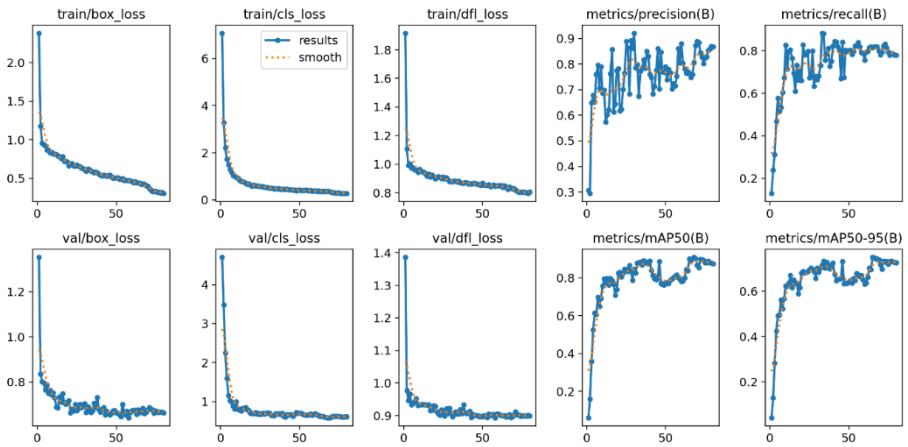


Figure 16: Train and Validation Results YOLOv11s model

Figure 16 shows the model achieved a mAP50-95 of 0.73 and mAP50 of 0.86 representing the mean average precision (mAP) at different Intersection of union (IoU) thresholds (0.5 = 50% overlap between predicted box and true box, 0.5-0.95 = 50%-95%). Demonstrating reasonable precision on band location and box placement.

Furthermore, the model achieved a precision of 0.87 representing how many box predictions were correct, and recall score of 0.78 demonstrating the models ability to find all true objects.

The resistance calculation function also performed well, scoring 100% correct resistance and tolerance values shown in Figure 15. The function is robust to different orientations of the resistor unless perfectly verticle, as this aligns box positions which generates error in band order and subsequent calculations.

The function is also limited to resistors that contain silver or gold bands, as these are used to appropriately orientate the resistor. Any resistor that does not contain these colours are subject to calculated inconsistencies dependant on its orientation.

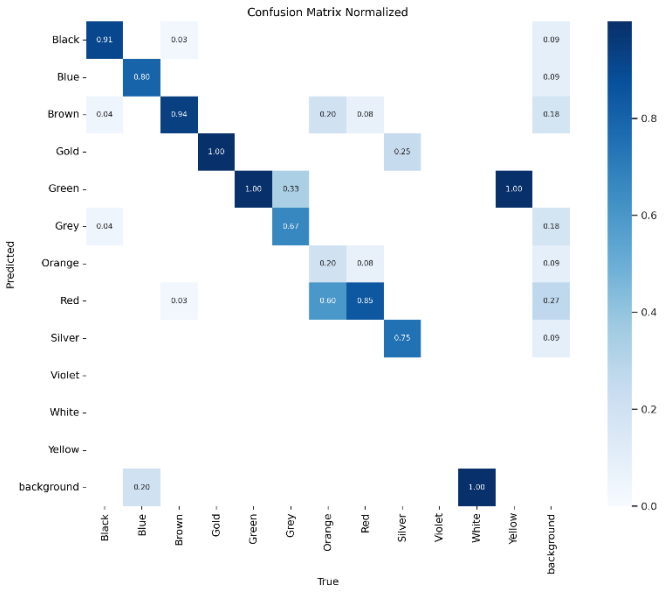


Figure 17: Normalised Confusion matrix results of the YOLO V11s model

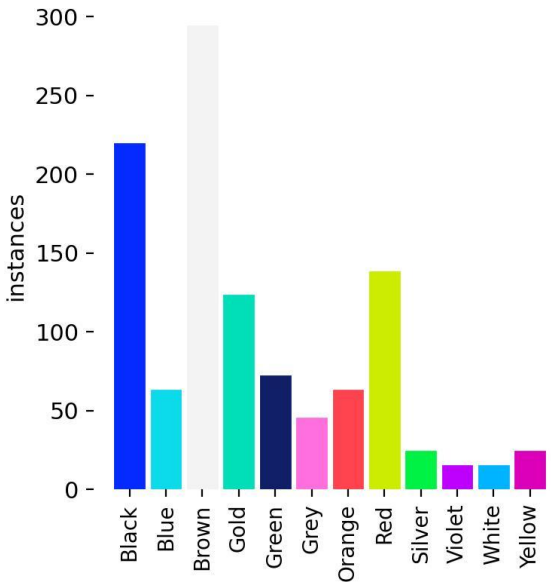


Figure 18: Label instances across the YOLO V11s models dataset

Due to colour similarities, lighting and reduced instances of particular colours within the dataset, the models colour accuracy on images outside similar conditions to the dataset can be mixed. Due to the low saturation of the dataset and dim lighting, similar colours such as red and orange are frequently confused due to likeness. This is represented in Figure 17 where orange was predicted red 60% of the time. However, this could also be due to labelling errors on the dataset during the contour drawing and labelling process.

Furthermore, variations in glare and colour saturation between the dataset and unseen images such as UWE dataset causes poor performance and reliability of the model, scoring only 1 out of 15 correct identifications and calculations (Figure 19).



Figure 19: Overall Algorithm Performance on UWE Dataset

6. Discussions and Conclusions

The overall algorithm provides a resistor identification accuracy of 0.97, mAP50-95 (B) of 0.73 and precision (B) of 0.87. With the resistor calculator having a 100% accuracy on unseen images based on its provided bands.

However, the algorithm does not generalise well in band identification due to the dataset being a majority of MIT (2024) which was too dark for real world applications. For this reason, band identification struggles with well lit, vivid, and colour rich environments. Similar colours such as red and orange are frequently confused, and band boxes are misplaced or unreliably generated. Human error may have played a role in misclassification as labelling is a long and tedious process which can result in labelling errors being passed to the model, affecting its accuracy.

Inconsistencies can also be found in the resistor Identification model where false negatives are occurrent. This is most likely due to a low epoch implementation of 6, preventing the model from adjusting weights further to improve accuracy.

Generalisability can be resolved through providing the models with an improved dataset. Focus should be towards more light diverse images, allowing for colours to be distinguished and identified better in brighter environments. Standardised labelling techniques should be used to reduce human error and an increase of images featuring under-represented classes such as violet, white and yellow should be added. This should improve the model's accuracy and reliability in detecting bands across diverse images, increasing the overall algorithms accuracy.

Further limitations are found within the resistor calculation function as its limited to 4-5 band resistors and prone to inaccuracy in specific orientations and band colours. To adjust for 6 band resistors, temperature coefficient and fail rate bands must be accounted for.

Rotation functions should also be implemented before sending images to the YOLO V11s model to ensure correct readings despite orientation. Creating a standardised orientation will prevent band centre overlap on the x-axis and allow accurate band readings. The furthest distance between the bands should also be found and used to determine the correct orientation through finding the tolerance band, preventing current reliance on gold or silver bands for reading directions.

Overall, the model performs well in select environments, but future work is needed to improve the accuracy and generalisability of the model. Focus should be on implementing mentioned improvements and experimenting with multi-resistor images to further improve the generalisability and accuracy of the algorithm in real world environments.

Reference List:

Alhanoof, A., Duaa, A., Heyam, A., Amani, S., Alanoud, B.D., Najla, A., Afnan Abou, E. and Heba, K. (2021) Impact of Dataset Size on Classification Performance: An Empirical Evaluation in the Medical Domain. *Applied Sciences* [online]. 11 (2), p. 796. [Accessed 21 April 2025].

Chen, D. (2025) *Week 5: Image Convolution* [lecture to Beng Robotics Year 2], UWE. 21 February 2025.

Fisher, R., Perkins, S., Walker, A. and Wolfart, E. (2003) *Median Filter*. Available from: https://homepages.inf.ed.ac.uk/rbf/HIPR2/median.htm [Accessed 26 March 2025].

Iqbal, S., Khan, T.M., Naveed, K., Naqvi, S.S. and Nawaz, S.J. (2022) Recent Trends and Advances in Fundus Image Analysis: A Review. *Computers in Biology and Medicine* [online]. Vol.151 (Pt A) (106277), p. 106277-106277. [Accessed 08 April 2025].

Javaid, M., Haleem, A., Singh, R.P., Rab, S. and Suman, R. (2022) Exploring Impact and Features of Machine Vision For Progressive Industry 4.0 Culture. *Sensors International* [online]. 3 (100132), p. 100132. [Accessed 20 April 2025].

KEA (2011) *Impulse Noise.* Available from: https://www.maths.nottingham.ac.uk/plp/pmzkz/saltpepper.html [Accessed 26 March 2025].

Kopparapu, S.K. (2006) Lighting Design For Machine Vision Application. *Image and Vision Computing* [online]. 24 (7), pp. 720-726. [Accessed 20 April 2025].

Liu, X., Song, L., Liu, S. and Zhang, Y. (2021) A Review of Deep-learning-based Medical Image Segmentation Methods. *Research on Sustainability and Artificial Intelligence* [online]. 13 (3), p. 1224. [Accessed 20 April 2025].

MIT (2024) *Resistors*. Available from: https://www.kaggle.com/datasets/barrettotte/resistors [Accessed 12 April 2025].

Muminovic, M. and Sokic, E. (2019) Automatic Segmentation and Classification of Resistors in Digital Images. In: 2019 XXVII International Conference on Information, Communication and Automation Technologies (ICAT). Sarajevo, Bosnia and Herzegovina, 20-23 October 2019. ICAT, p.1-6.

Powell, V. (2015) *Image Kernels Explained Visually.* Available from: https://setosa.io/ev/image-kernels/#:~:text=The%20sharpen%20kernel%20emphasizes%20differences,differences%20in%20adjacent%20pixel%20values [Accessed 27 March 2025].

Trigka, M. and Dritsas, E. (2025) A Comprehensive Survey of Deep Learning Approaches in Image Processing. *Sensors; Basel* [online]. 25 (2), p. 531. [Accessed 20 April 2025].

UK Photo and Film Archive (2023) *Portraits of First World War Soldiers.* Available from: https://www.ukphotoarchive.org.uk/portraits-of-ww1-soldiers [Accessed 26 March 2025]

Varna, D. and Abromavičius, V. (2022) A System For a Real-time Electronic Component Detection and Classification on a Conveyor Belt. *Applied Sciences; Basel* [online].12 (11), p. 12115608. [Accessed 20 April 2025].

Zhou, L., Zhang, L. and Konz, N. (2023) Computer Vision Techniques in Manufacturing. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* [online]. 53 (1), pp. 105-117. [Accessed 20 April 2025].

42Electronics (2025) *Calculating Resistor Value*. Available from: https://42electronics.com/blogs/learn-more/calculating-resistor-value [Accessed 24 April 2025]